
Livello di rete

Parte 3

Internet, Web e Cloud
Claudia Canali



Soluzione Esercizio

Soluzione Esercizio



- Si consideri una rete con un router avente 4 collegamenti in uscita e che i pacchetti debbano essere inoltrati sulle interfacce di uscita come segue

Intervallo Indirizzi Destinazione	Interfaccia di collegamento
da 11100000 00000000 00000000 00000000 a 11100000 00111111 11111111 11111111	0
da 11100000 01000000 00000000 00000000 a 11100000 01000000 11111111 11111111	1
da 11100000 01000001 00000000 00000000 a 11100000 01111111 11111111 11111111	2
altrimenti	3

**Scrivere una tabella di
inoltro di 4 righe che
usa il confronto a
prefisso più lungo**

Soluzione Esercizio



- Si consideri una rete con un router avente 4 collegamenti in uscita e che i pacchetti debbano essere inoltrati sulle interfacce di uscita come segue

Intervallo Indirizzi Destinazione		Interfaccia di collegamento
da	11100000 00000000 00000000 00000000	0
a	11100000 00111111 11111111 11111111	
da	11100000 01000000 00000000 00000000	1
a	11100000 01000000 11111111 11111111	
da	11100000 01000001 00000000 00000000	2
a	11100000 01111111 11111111 11111111	
altrimenti		3

Scrivere una tabella di inoltrato di 4 righe che usa il confronto a prefisso più lungo

Tabella di inoltrato		
11100000	00	0
11100000	01000000	1
11100000	01	2
altrimenti		3

Esercizio



- Descrivere come la vostra tabella di inoltro determini l'interfaccia di collegamento corretta per i datagrammi con i seguenti indirizzi

D1 11001000 10010001 01010001 01010101 → 3

D2 11100000 01000000 11000011 00111100 → 1

D3 11100000 01000001 00010001 01110111 → 2

Tabella di inoltro	
11100000 00	0
11100000 01000000	1
11100000 01	2
altrimenti	3



Routing IP

Next-hop routing



- Abbiamo detto come il forwarding IP (inoltro) è basato sul concetto di **next-hop routing**
- **Indipendenza dal mittente**: il **next-hop routing** non dipende (tipicamente) dal mittente del pacchetto o dal cammino che il pacchetto ha attraversato fino a quel momento
 - Basato fondamentalmente sull'indirizzo del destinatario
- **Routing universale**: la tabella di routing deve contenere un **next-hop router** per ciascuna destinazione (interfaccia di uscita)
- **Routing ottimo**: il next-hop router è scelto in modo da minimizzare il cammino verso la destinazione → **determinato a priori dagli algoritmi di routing**

- Internet non è un insieme di router “sparsi” casualmente nel mondo che vengono interconnessi tra di loro
 - I router sono aggregati in **regioni, chiamate Autonomous Systems (AS)**
- Definizione

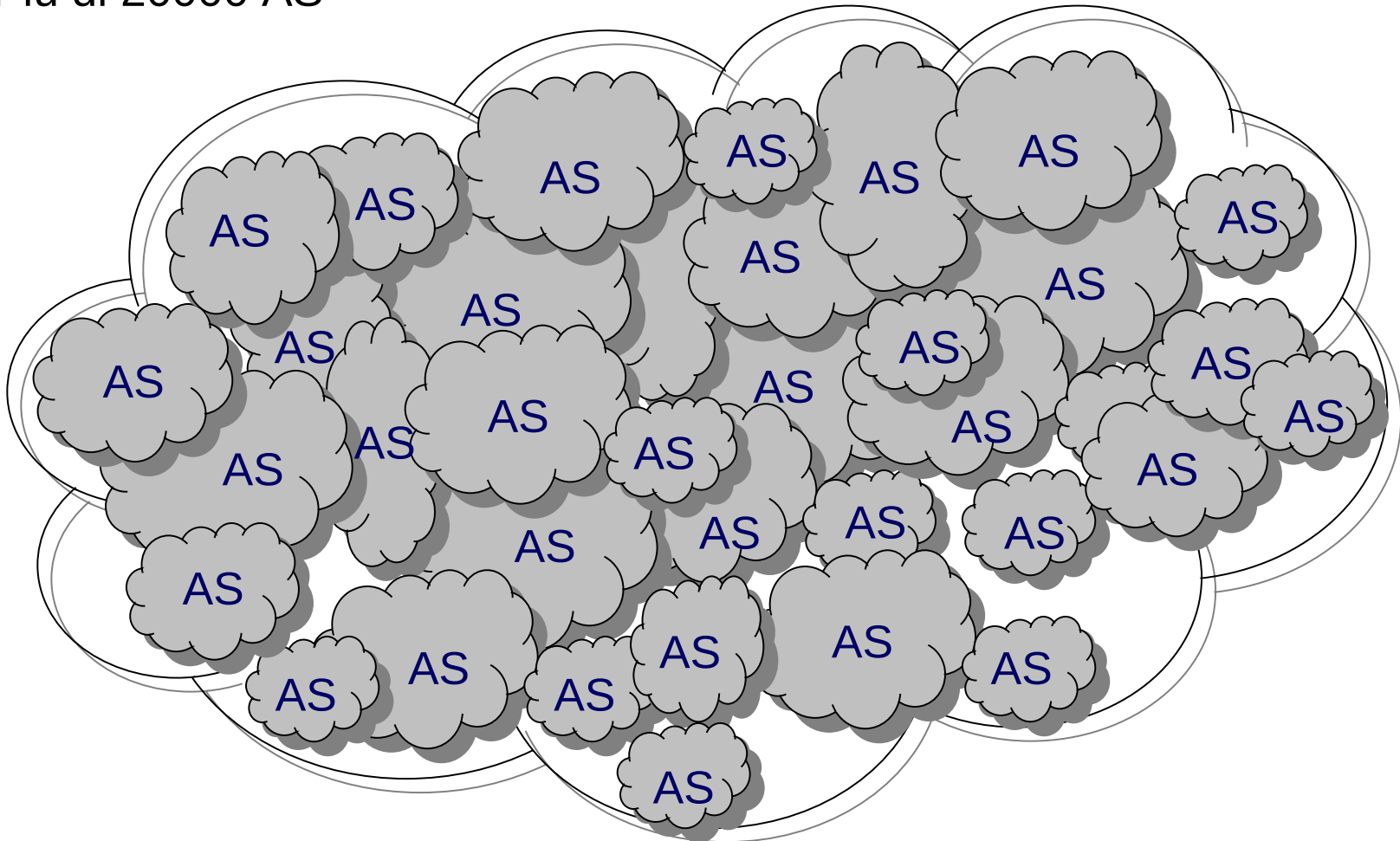
“Un insieme di reti IP (network prefix) e di router sotto il controllo di una organizzazione (o consorzio di) nell’ambito del quale si utilizza una politica di interior routing. Gli AS sono le unità delle politiche di exterior routing, come nel caso del BGP” [RFC 1930]

Autonomous Systems



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Più di 20000 AS



- Non tutti i router sono uguali per potenza, per configurazione e modalità di azione
- Tuttavia, tutti i router all'interno dello stesso **AS** usano lo **stesso algoritmo di instradamento dei messaggi (routing)** e si **scambiano continue informazioni con gli altri router**
- Gli Autonomous Systems dall'esterno vengono visti come un'**unica entità**
- **NOTA IMPORTANTE:**
 - Nessun AS gestisce più del 5% del traffico
 - La stragrande maggioranza degli AS gestisce **molto meno dell'1% del traffico**

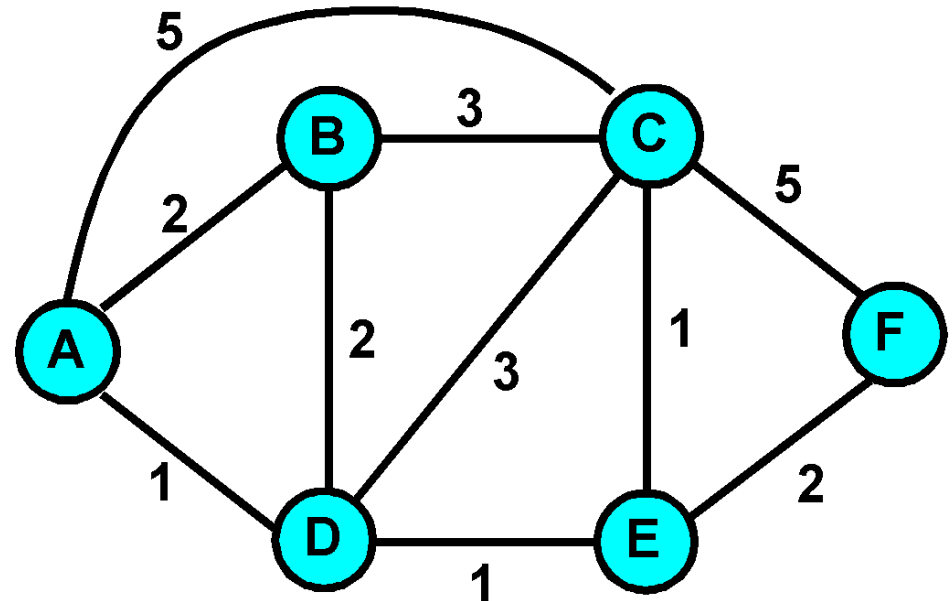
- **OBIETTIVO:** Determinare il percorso ottimale
- Dato un insieme di router interconnessi, determinare il cammino ottimale dall'host mittente all'host destinatario
- **Cammino ottimale** → **costo minimo (least-cost path)**
- Per formulare un algoritmo di routing, si modella la rete tramite un **grafo pesato $G(N,E)$** dove:
 - i nodi N rappresentano i router (oppure gli Autonomous Systems)
 - gli archi rappresentano le connessioni tra i router
 - le etichette E degli archi rappresentano il “costo” delle connessioni tra i router
- Solo se tutti gli archi hanno lo stesso costo, il cammino ottimale è anche il più corto (shortest-path)

Algoritmi di routing

Il costo di un collegamento può riflettere:

- lunghezza fisica del collegamento (link transoceanico più costoso)
- velocità del collegamento
- 'prezzo' del collegamento

Etichetta sull'arco:
"costo" (tempo) per
l'invio di un pacchetto



Qual è il cammino ottimale (e il suo costo) tra A e F?
Qual è il cammino ottimale (e il suo costo) tra A e C?

Fattori che influenzano il routing

- **Fattori statici:** topologia della rete
- **Fattori dinamici:** traffico della rete, guasti
- **Politiche di routing**

Principali algoritmi di routing

- **Algoritmi di routing centralizzato**
 - Ogni nodo possiede un'informazione globale sulla rete
 - Input dell'algoritmo: tutti i collegamenti tra nodi e relativi costi
 - Controller centralizzato o calcolo replicato in ogni router della rete
 - Es., Link State (LS) protocol
- **Algoritmi di routing distribuito**
 - Nessun nodo ha un'informazione completa del costo di tutti i link
 - Percorso calcolato in modo distribuito e iterativo: iniziale conoscenza dei collegamenti diretti poi scambio di informazioni con nodi adiacenti per calcolare gradualmente il percorso verso le destinazioni
 - Es., Distance Vector (DV) protocol

Algoritmi Link State

- **Gli algoritmi Link State (LS) sono centralizzati**
- Prevedono che la topologia di rete e i costi di ogni link siano noti (disponibili in input all'algoritmo):
 - Ogni nodo calcola lo stato dei link ad esso connessi
 - Ciascun nodo periodicamente trasmette identità e costi dei link connessi (**link state broadcast**)
 - *Quindi tutti i nodi hanno una visione identica e completa della rete*
 - Ciascun nodo calcola i cammini di costo minimo verso tutti gli altri nodi della rete mediante l'**Algoritmo di Dijkstra**

Informazioni sullo stato dei link



- Periodicamente vengono inviati in broadcast, su tutti i link del nodo, dei pacchetti **LSP (Link State Packet)** con le seguenti informazioni:
 - Node ID
 - Lista di vicini e costo dei rispettivi link
 - Informazioni aggiuntive:
 - **Numero di sequenza** per accorgersi di errori in caso di delivery out-of-order delle informazioni
 - **Time To Live (TTL)** per evitare di usare informazioni vecchie e quindi non affidabili

Propagazione pacchetti LSP



- Inoltro con algoritmo di flooding (inondazione)
- Quando il **nodo i riceve un LSP dal nodo j**:
 - Se il pacchetto LSP è il **più recente** proveniente da j, viene salvato nel database e una copia viene inoltrata su tutti i link connessi al nodo i (ad eccezione di quello da cui l'LSP è stato ricevuto)
 - Altrimenti il pacchetto LSP viene **scartato**
- **Alla fine degli scambi tutti i nodi disporranno di una visione identica e completa della rete**
- **A questo punto entra in azione l'algoritmo di Dijkstra: ogni nodo effettua il calcolo per trovare i cammini minimi da se stesso a tutti gli altri nodi della rete**

Forward search algorithm (Dijkstra)



- **Algoritmo iterativo:** alla k-esima iterazione, il nodo i conosce il cammino di costo minore definitivo verso k nodi destinazione
- Ogni nodo (router) esegue l'algoritmo
- Si definiscono:
 - **$c(i,j)$** costo del link tra nodo i e nodo j
 - **$D(v)$** costo minimo del cammino verso il nodo v (costo minimo temporaneo → per la iterazione corrente)
 - **$p(v)$** immediato predecessore di v lungo il cammino a costo minimo verso v
- **N** sottoinsieme di nodi il cui cammino di costo minore è noto definitivamente

Algoritmo di Dijkstra



- **Passo di inizializzazione seguito da un ciclo eseguito una volta per ogni nodo del grafo**
- **Al termine l'algoritmo avrà calcolato i cammini minimi dal nodo origine u verso tutti gli altri nodi**
- **Algoritmo di Dijkstra eseguito dal nodo (origine) u**

Inizializzazione

$N = \{u\}$

Per tutti i nodi v

se v è adiacente a u

$$D(v) = c(u,v)$$

altrimenti $D(v) = \infty$

Algoritmo di Dijkstra



Ciclo

- Determina un w **non in N** tale che il costo $D(w)$ sia **minimo dei costi verso gli altri nodi non in N**
- **Aggiungi a N il nodo w** con il costo minimo $D(w)$
- **Aggiorna $D(v)$ per ciascun nodo v adiacente a w e non in N :**

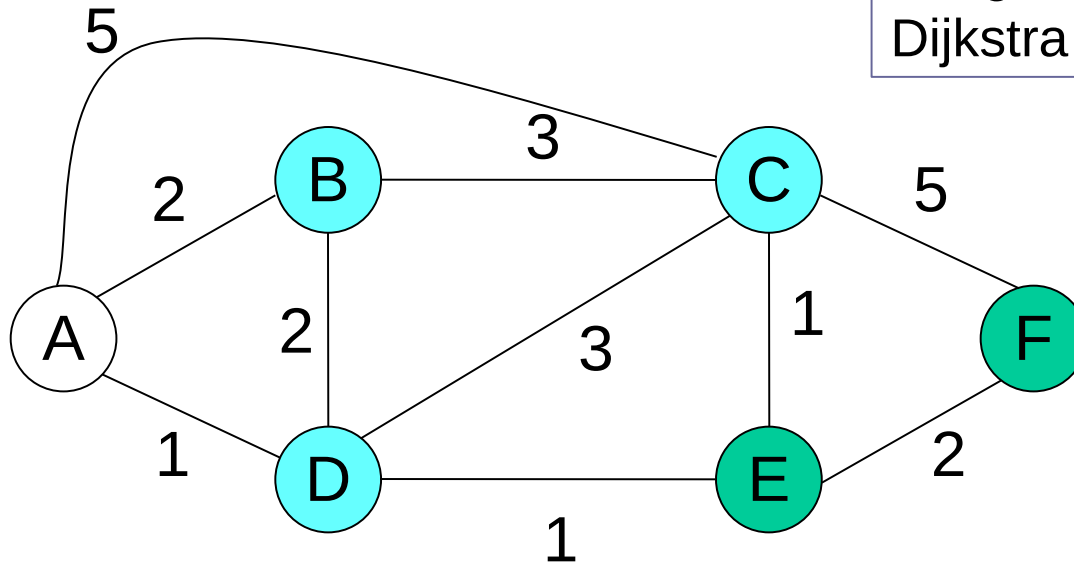
$$D(v) = \min\{ D(v), D(w) + c(w,v) \}$$

- **Until** tutti i nodi del grafo sono nell'insieme N

Il nuovo costo verso v è il vecchio costo verso v o il costo del cammino minimo verso w più il costo da w a v

Esempio (step 1)

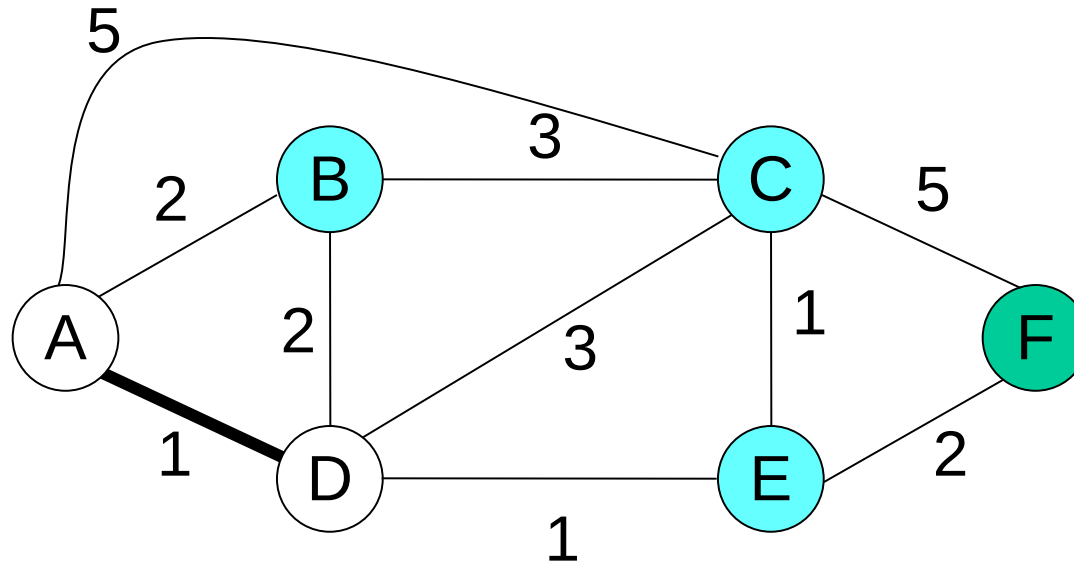
Eseguiamo l'algoritmo di
Dijkstra per il nodo A



Passo	N	D(B), p(B)	D(C), p(C)	D(D), p(D)	D(E), p(E)	D(F), p(F)
1	A	2, A	5, A	1, A	∞	∞



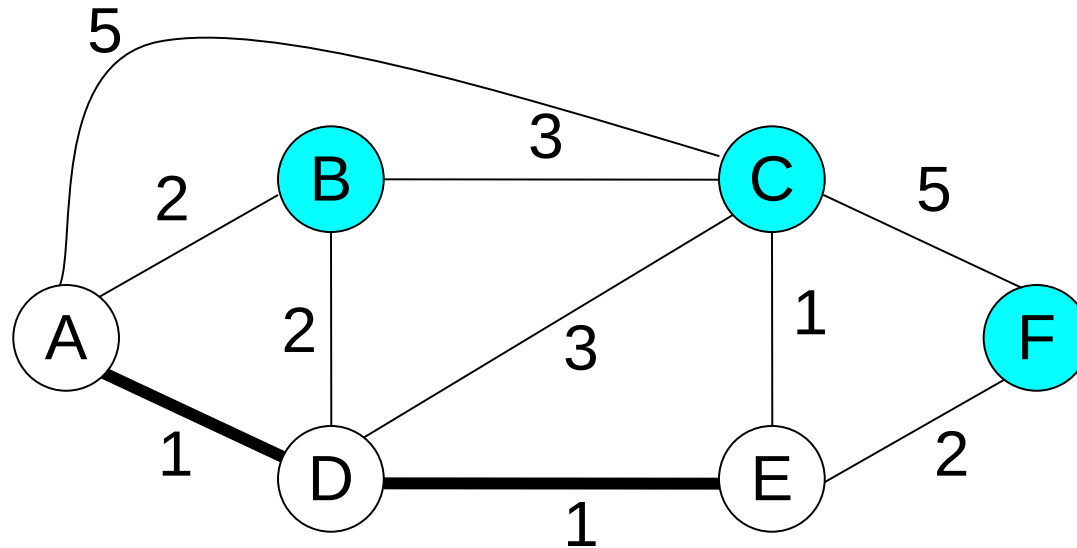
Esempio (step 2)



Passo	N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞



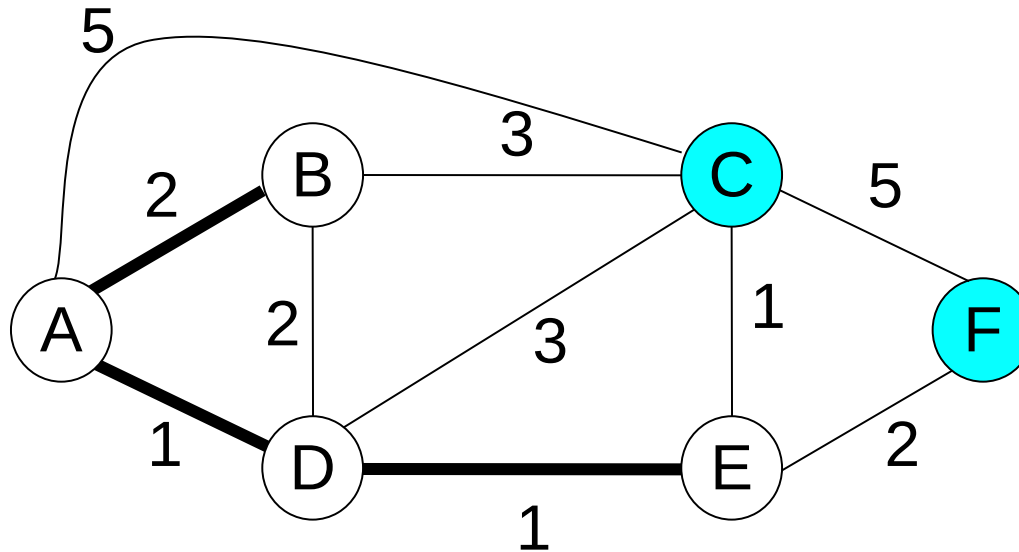
Esempio (step 3)



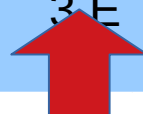
Passo	N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	
3	ADE	2,A	3,E			4,E



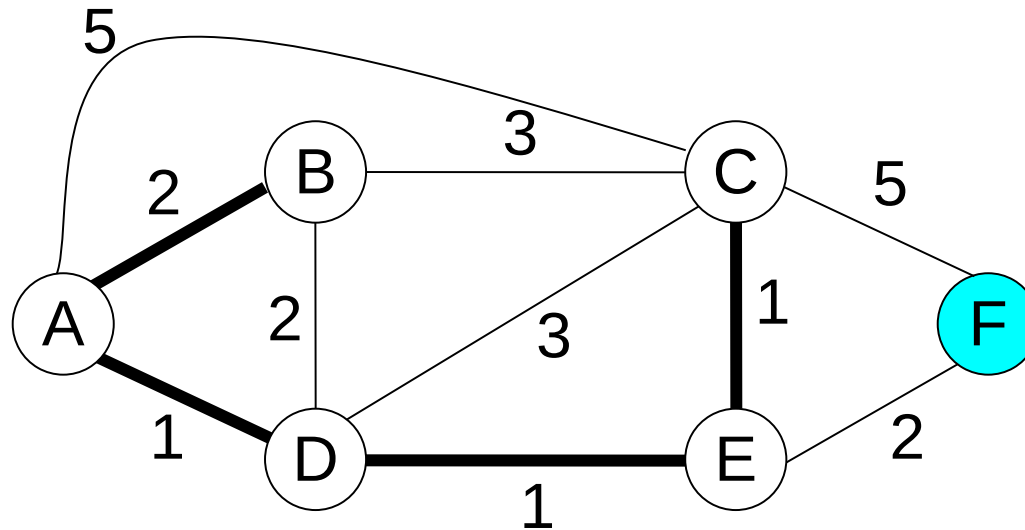
Esempio (step 4)



Passo	N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	
3	ADE	2,A	3,E			4,E
4	ADEB		3,E			4,E

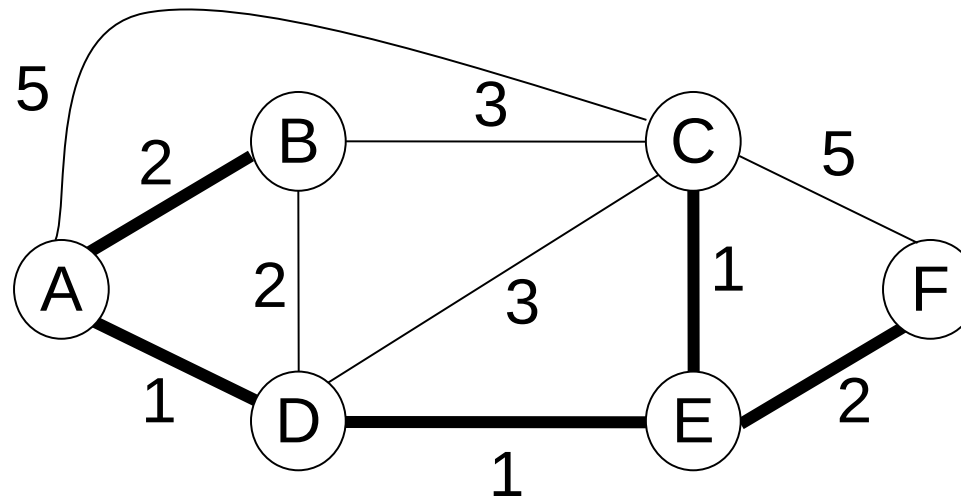


Esempio (step 5)



Passo	N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	
3	ADE	2,A	3,E			4,E
4	ADEB		3,E			4,E
5	ADEBC					4,E

Esempio (step 6)



Passo	N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	
3	ADE	2,A	3,E			4,E
4	ADEB		3,E			4,E
5	ADEBC					4,E



Algoritmi Distance Vector

Algoritmi Distance Vector



- Usati nel **primo periodo di Internet** (ARPANET)
- **Calcolo distribuito** del next hop
 - Ciascun nodo riceve un'informazione parziale sulla rete da uno o più dei suoi vicini direttamente connessi, a cui, dopo aver effettuato il calcolo/aggiornamento, restituisce i risultati sulle proprie distanze
- Unità di **scambio** dell'informazione:
 - **Vettore di distanze** rispetto alle varie destinazioni
- Algoritmo **adattativo** rispetto a cambi di stato (non periodico)
- Esempio principale di implementazione:
 - **Algoritmo Bellman-Ford distribuito**

Algoritmo Bellman-Ford: premessa



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Ogni nodo:

- **aggiorna** il proprio **vettore di distanze** in risposta a variazioni di costi sui link adiacenti
- invia un **aggiornamento ai nodi adiacenti** se il proprio vettore di distanze cambia

Ogni nodo x mantiene i seguenti dati:

- $c(x,v)$ costo del link tra nodo x e nodo v
- $Dx=[Dx(y): y \in N]$ vettore di distanze del nodo x verso tutti i nodi y nella rete N
- $Dv=[Dv(y): y \in N]$ vettori di distanze dei vicini v di x

Algoritmo Bellman-Ford: premessa



- Uso della **formula di Bellman-Ford** per il calcolo del costo minimo tra x e y :
 - $Dx(y) = \min_v \{ c(x,v) + Dv(y) \}$
dove $\min_v$ è calcolato tra tutti i vicini v del nodo x
- **Intuitivamente**, la formula è molto chiara:
 - Se dopo aver viaggiato da x a v , consideriamo il percorso a costo minimo da v a y , il costo del percorso intero sarà $c(x,v) + Dv(y)$
 - Dato che dobbiamo per forza iniziare il percorso viaggiando verso qualche vicino v , il costo minimo da x a y è il minimo di $(c(x,v) + Dv(y))$ calcolato su tutti i nodi v adiacenti a x

Algoritmo Bellman-Ford



Start (fase iniziale di configurazione della rete)

Per ogni destinazione y in N :

$Dx(y) = c(x,y)$ se y è adiacente

$Dx(y) = \infty$ se y non è adiacente

Invia il vettore di distanze $Dx = [Dx(y) \mid y \text{ in } N]$ ad ogni vicino v

Loop (fase a regime)

Attendi (finchè il costo di un collegamento verso qualche vicino v cambia o ricevi un vettore di distanze da un vicino v), poi:

Per ogni destinazione y in N :

$Dx(y) = \min_v \{ c(x,v) + Dv(y) \}$

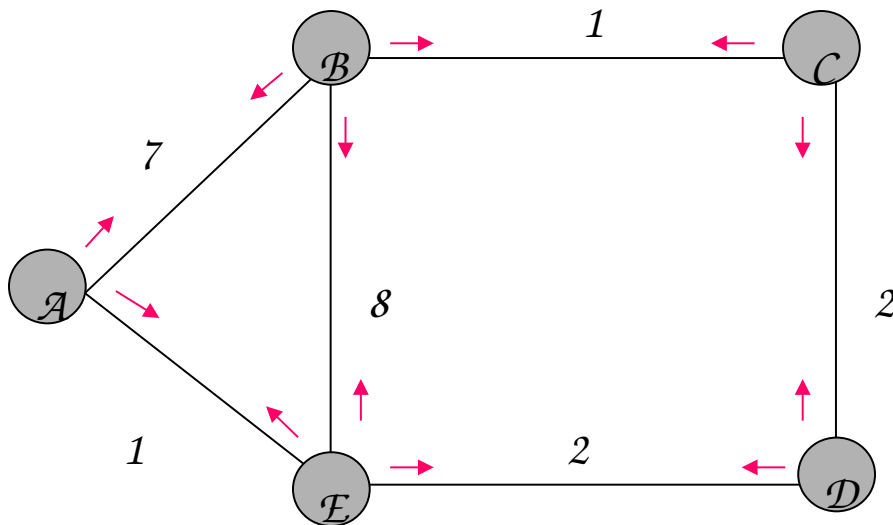
Se $Dx(y)$ è cambiato per qualche destinazione y

invia il vettore di distanze $Dx = [Dx(y) : y \text{ in } N]$ ad ogni vicino v

Esempio: distanze iniziali (start)

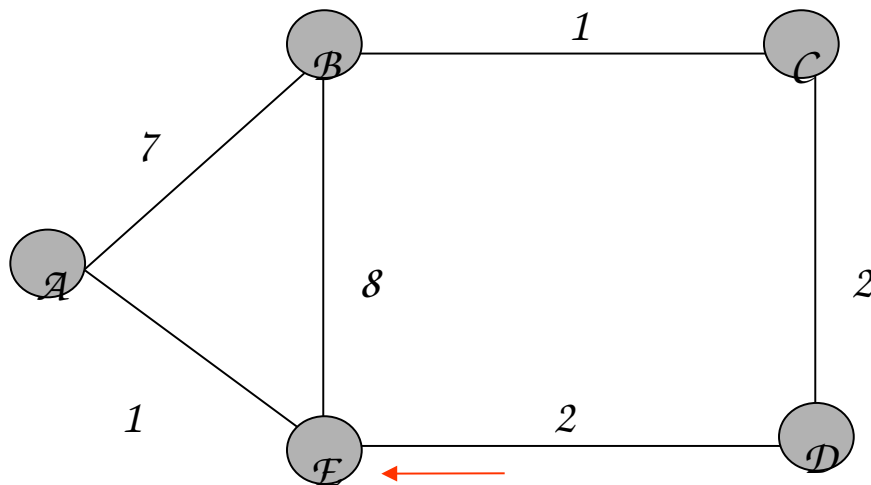


Per ogni nodo: calcolo delle distanze e invio ai vicini



<i>Distanza dal nodo</i>					
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>	0	7	∞	∞	1
<i>B</i>	7	0	1	∞	8
<i>C</i>	∞	1	0	2	∞
<i>D</i>	∞	∞	2	0	2
<i>E</i>	1	8	∞	2	0

E riceve il vettore delle distanze da D

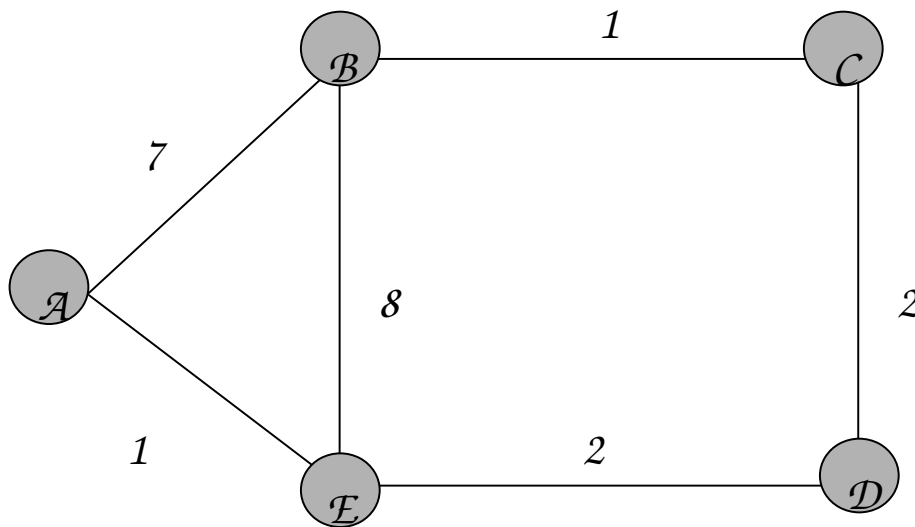


	<i>Distanza dal nodo</i>				
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>	0	7	∞	∞	1
<i>B</i>	7	0	1	∞	8
<i>C</i>	∞	1	0	2	∞
<i>D</i>	∞	∞	2	0	2
<i>E</i>	1	8	∞	2	0

Ora E può aggiornare le proprie distanze verso gli altri nodi della rete



E aggiorna i costi per C

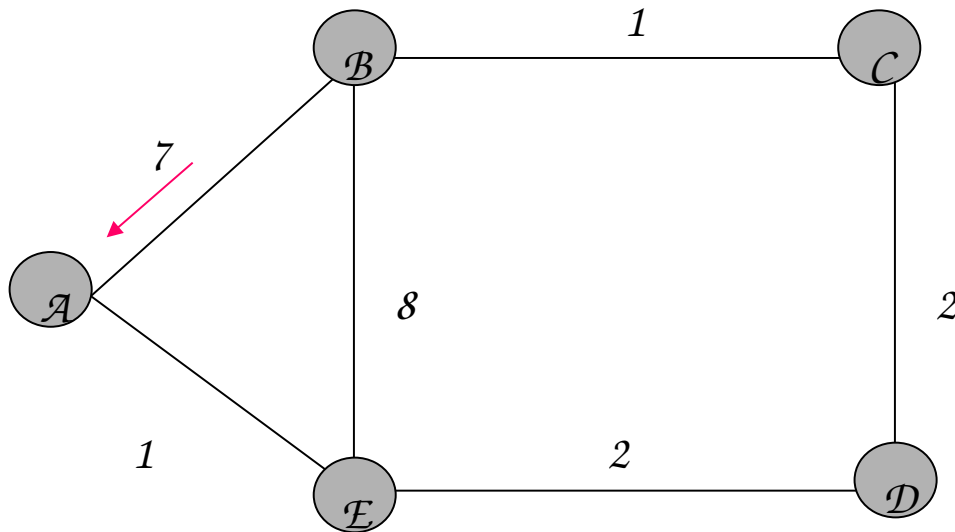


		<i>Distanza dal nodo</i>				
		<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>		0	7	∞	∞	1
<i>B</i>		7	0	1	∞	8
<i>C</i>		∞	1	0	2	∞
<i>D</i>		∞	∞	2	0	2
<i>E</i>		1	8	4	2	0

Cambia la distanza verso C



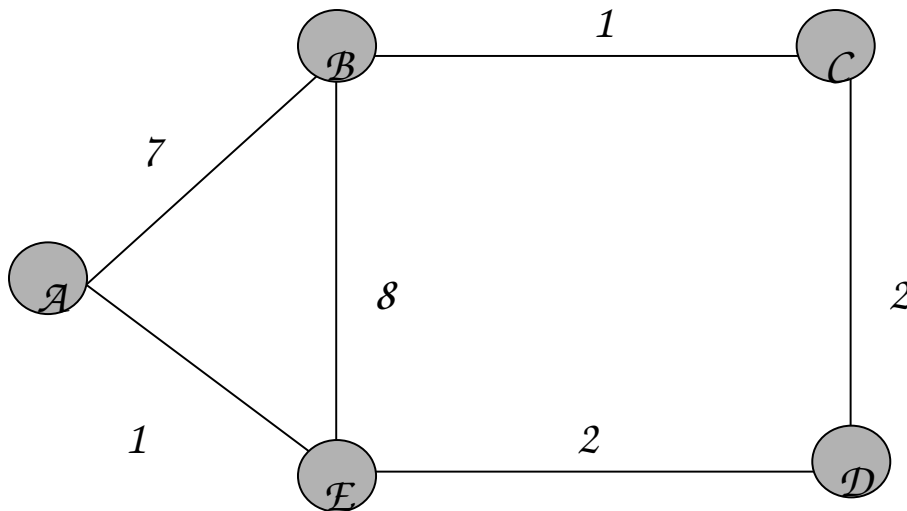
A riceve il vettore delle distanze da B



		<i>Distanza dal nodo</i>				
		<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>		0	7	∞	∞	1
<i>B</i>		7	0	1	∞	8
<i>C</i>		∞	1	0	2	∞
<i>D</i>		∞	∞	2	0	2
<i>E</i>		1	8	4	2	0

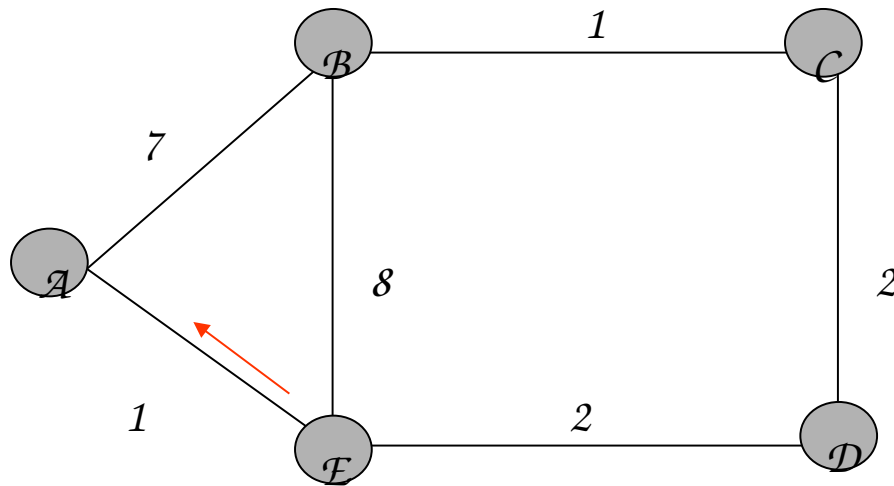
Ora A può aggiornare le proprie distanze verso gli altri nodi della rete

A aggiorna i costi per C



		<i>Distanza dal nodo</i>				
		<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>		0	7	8	∞	1
<i>B</i>		7	0	1	∞	8
<i>C</i>		∞	1	0	2	∞
<i>D</i>		∞	∞	2	0	2
<i>E</i>		1	8	4	2	0

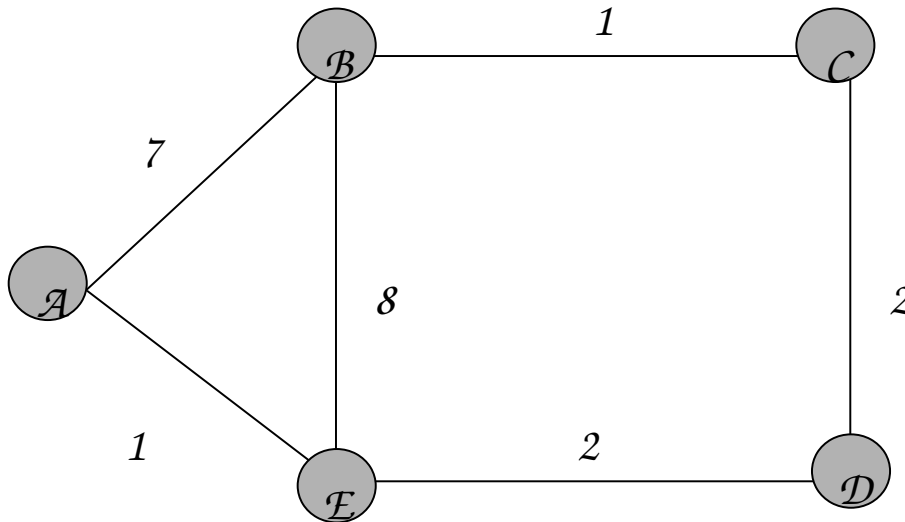
A riceve il vettore delle distanze da E



		Distanza dal nodo				
		A	B	C	D	E
A		0	7	8	∞	1
B		7	0	1	∞	8
C		∞	1	0	2	∞
D		∞	∞	2	0	2
E		1	8	4	2	0

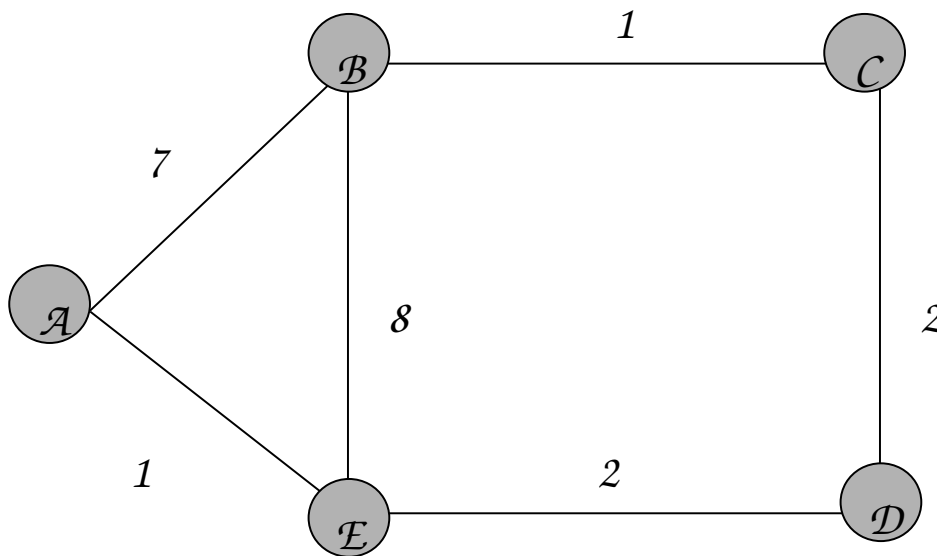
Ora A può aggiornare le proprie distanze verso gli altri nodi della rete

A aggiorna i costi per C e D



<i>Distanza dal nodo</i>					
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>	0	7	5	3	1
<i>B</i>	7	0	1	∞	8
<i>C</i>	∞	1	0	2	∞
<i>D</i>	∞	∞	2	0	2
<i>E</i>	1	8	4	2	0

Distanze finali aggiornate



		<i>Distanza dal nodo</i>				
		<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>		0	6	5	3	1
<i>B</i>		6	0	1	3	5
<i>C</i>		5	1	0	2	4
<i>D</i>		3	3	2	0	2
<i>E</i>		1	5	4	2	0

NOTA: quella mostrata è una tabella riassuntiva finale di tutti i cammini della rete. Nella pratica, ogni nodo tiene traccia di tutti i calcoli precedenti (non solo dei cammini di costo minimo) nella sua tabella di routing interna

Osservazione

- Instradamento **Link State**: è **centralizzato** perché ogni nodo deve avere l'informazione completa sulla rete prima di mandare in esecuzione l'algoritmo di Dijkstra
- Instradamento **Distance Vector** è **decentralizzato** e non usa tale informazione globale
- Le sole informazioni ritenute dai nodi sono:
 - Il **costo del collegamento verso i vicini** direttamente connessi
 - Le **informazioni ricevute dai vicini** (usate per ricalcolo)
 - Il **proprio vettore delle distanze** dagli altri nodi della rete
- Ogni nodo attende aggiornamenti dai vicini e in base a quelli aggiorna il proprio vettore delle distanze e (se cambiato) lo distribuisce ai nodi vicini

Tabella di routing



- L'algoritmo di Bellman-Ford ha un'immediata ricaduta pratica. Serve, infatti, per calcolare i **valori della Tabella di routing** (tabella di instradamento) di ciascun nodo
- La tabella di routing è **diversa** dalla tabella di inoltro
 - *La tabella di routing **serve per calcolare** la tabella di inoltro (capiremo meglio nel routing geografico)*
- La **Tabella di routing del nodo x** ha:
 - una riga per ogni nodo destinazione nella rete considerata (nodo considerabile come router o AS)
 - tante colonne quanti sono i nodi adiacenti al nodo x
 - Indicazione del next-hop sul cammino
 - i costi di cammino come elementi della tabella

Tabella di routing di E

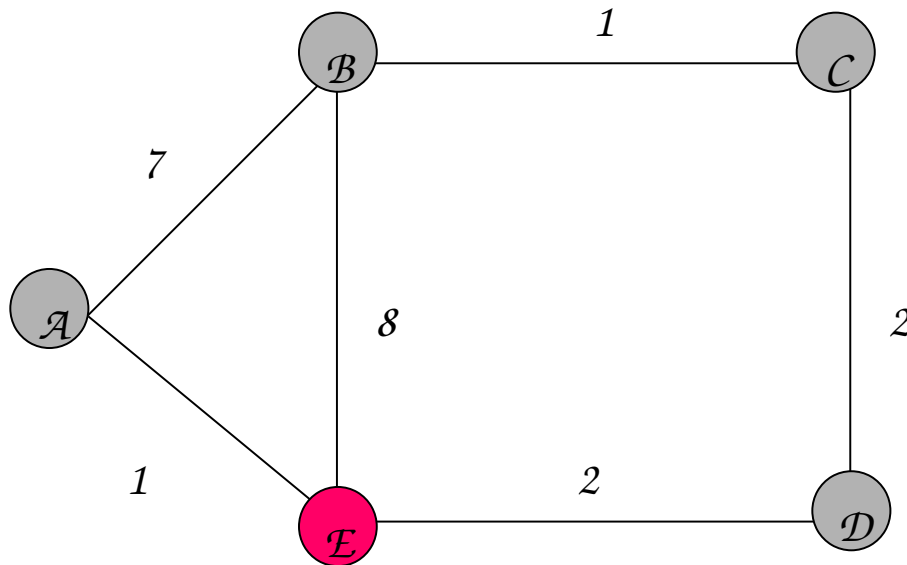


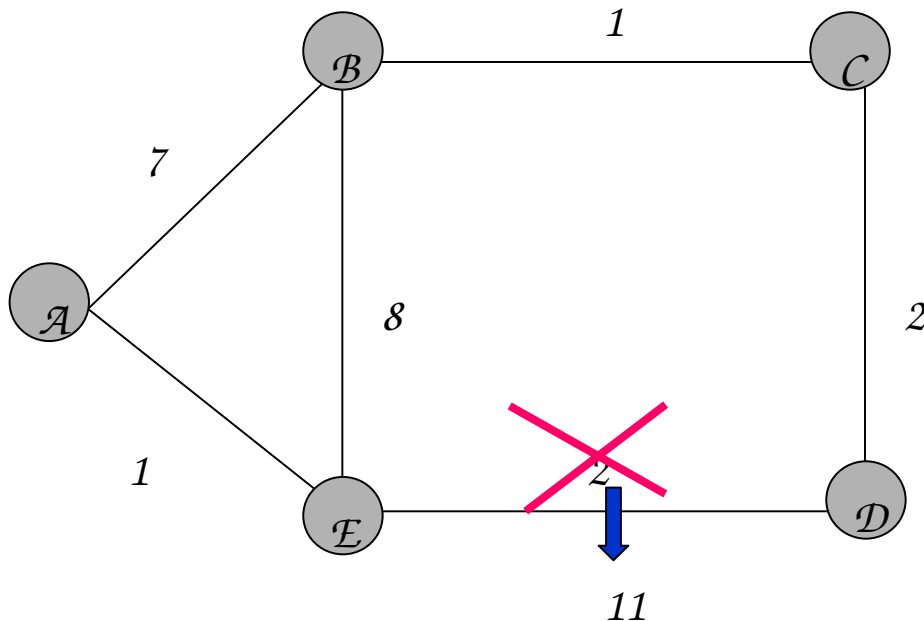
Tabella di routing di E

Dest	Next hop		
	A	B	D
A	1	14	5
B	8	8	5
C	6	9	4
D	4	11	2

La routing table di E ha una riga per ogni destinazione nella rete e tante colonne quanti sono i nodi adiacenti al nodo stesso (next hop sul cammino)

I percorsi di minor costo per la corrispondente destinazione sono indicati in rosso nella *routing table*

DV: modifica costi e problemi nei collegamenti



		<i>Distanza dal nodo</i>				
		<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>		0	6	5	3	1
<i>B</i>		6	0	1	3	5
<i>C</i>		5	1	0	2	4
<i>D</i>		3	3	2	0	2
<i>E</i>		1	5	4	2	0

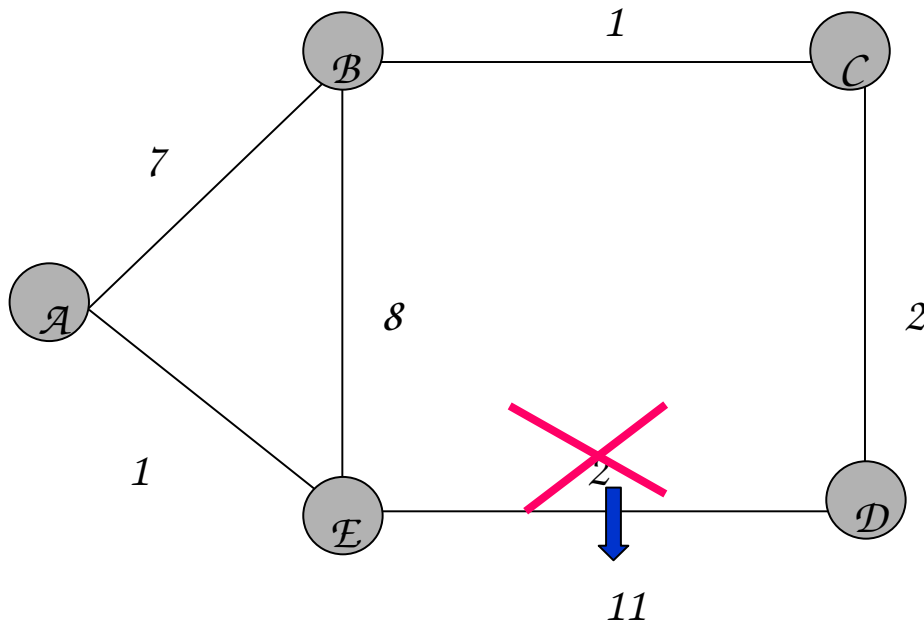
Ipotesi: il link E-D ha dei problemi – costo passa da 2 a 11

I nodi che vertono su quel link (D e E) ricalcolano il vettore distanza

Aggiornano la propria routing table e trasmettono il nuovo vettore ai vicini

Ciascun nodo ricalcolerà il proprio vettore distanza e, iterativamente, lo invierà ai nodi vicini (considerate ad esempio il nodo C)

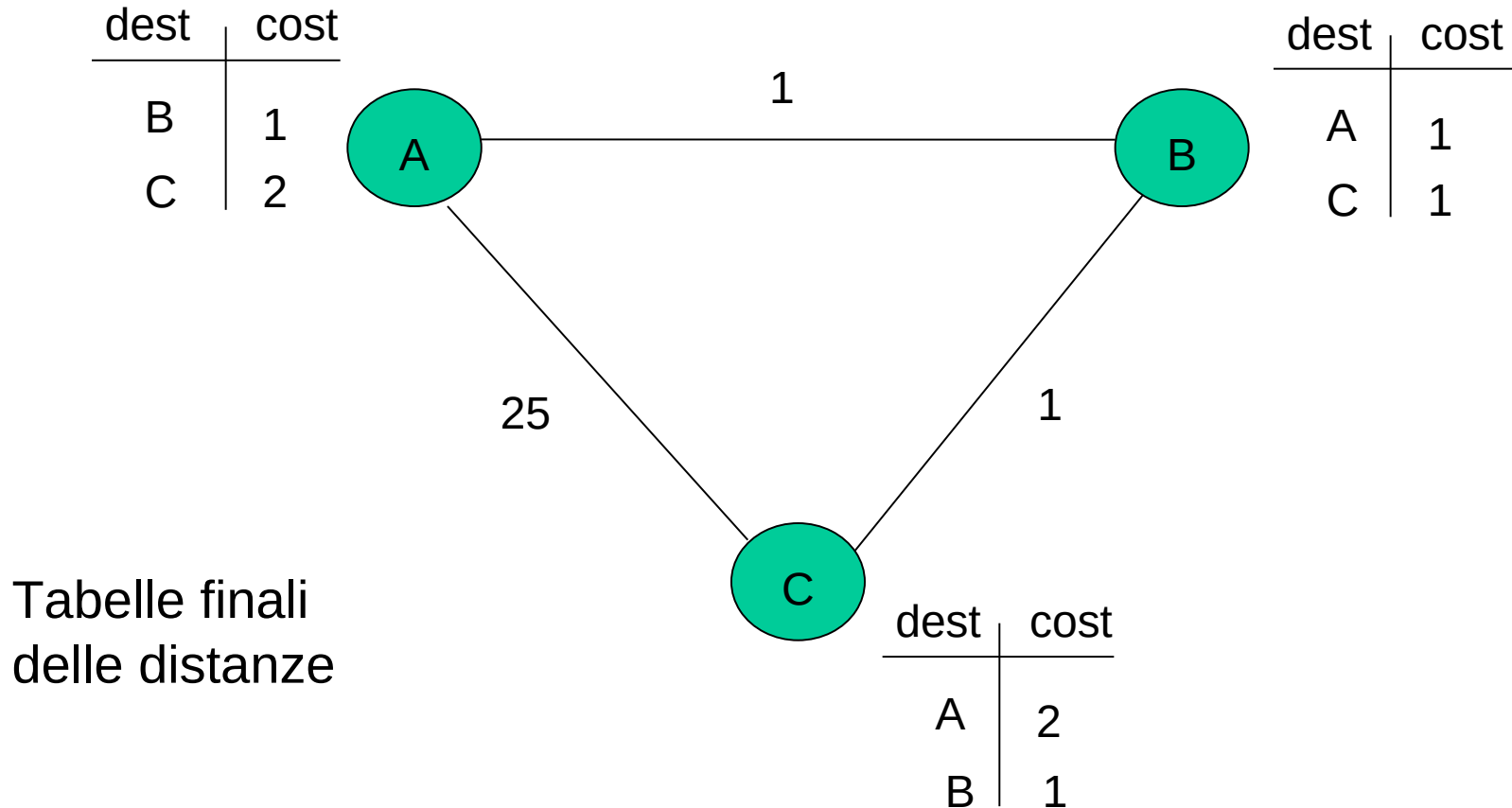
DV: modifica costi e problemi nei collegamenti



		<i>Distanza dal nodo</i>				
		<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>		0	7	8	10	1
<i>B</i>		7	0	1	3	8
<i>C</i>		8	1	0	2	9
<i>D</i>		10	3	2	0	11
<i>E</i>		1	8	9	11	0

Tabella aggiornata finale in base al cambiamento sul link E-D

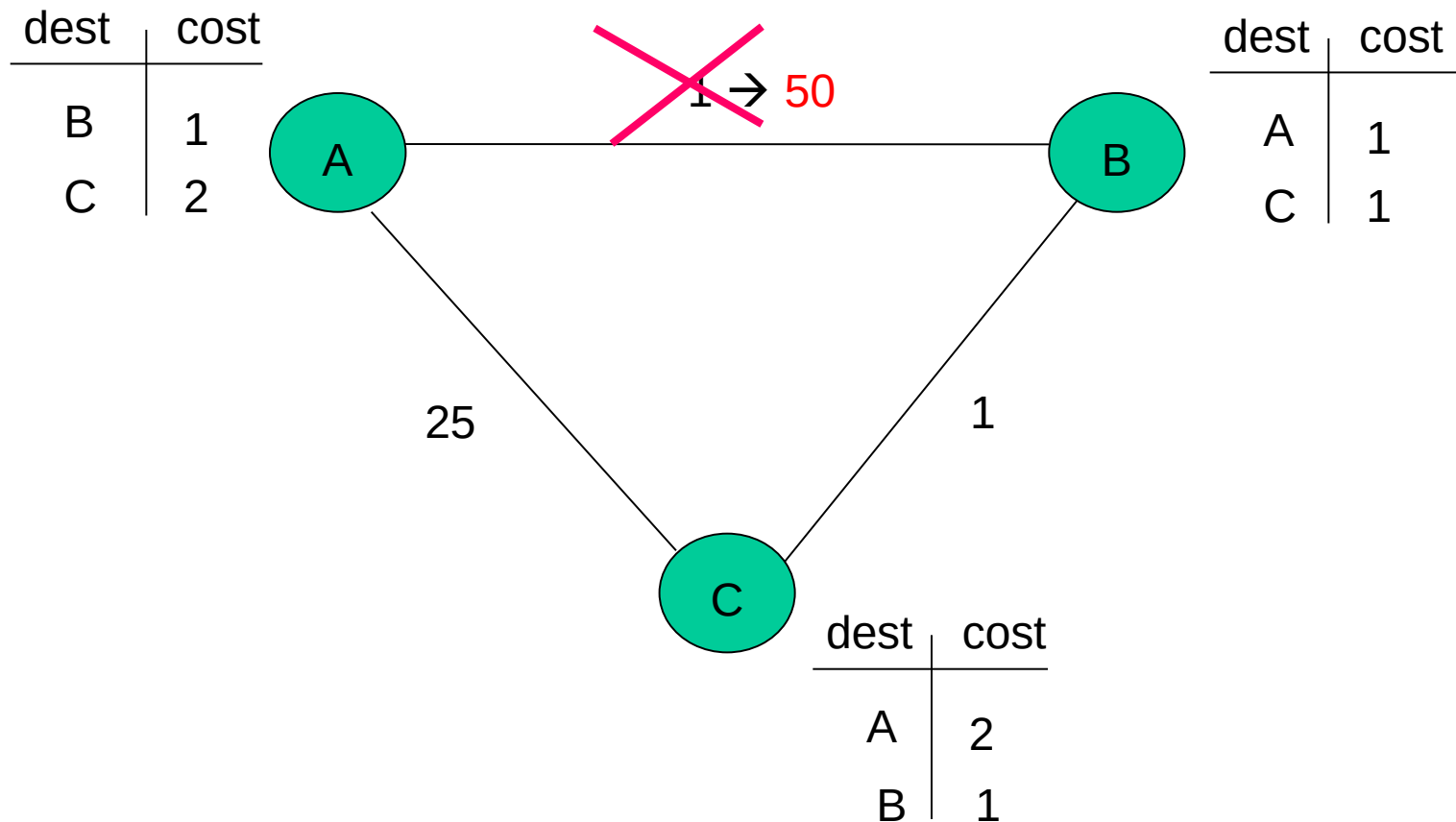
Rischio Bellman-Ford: effetto rimbalzo



Link A-B ha un problema



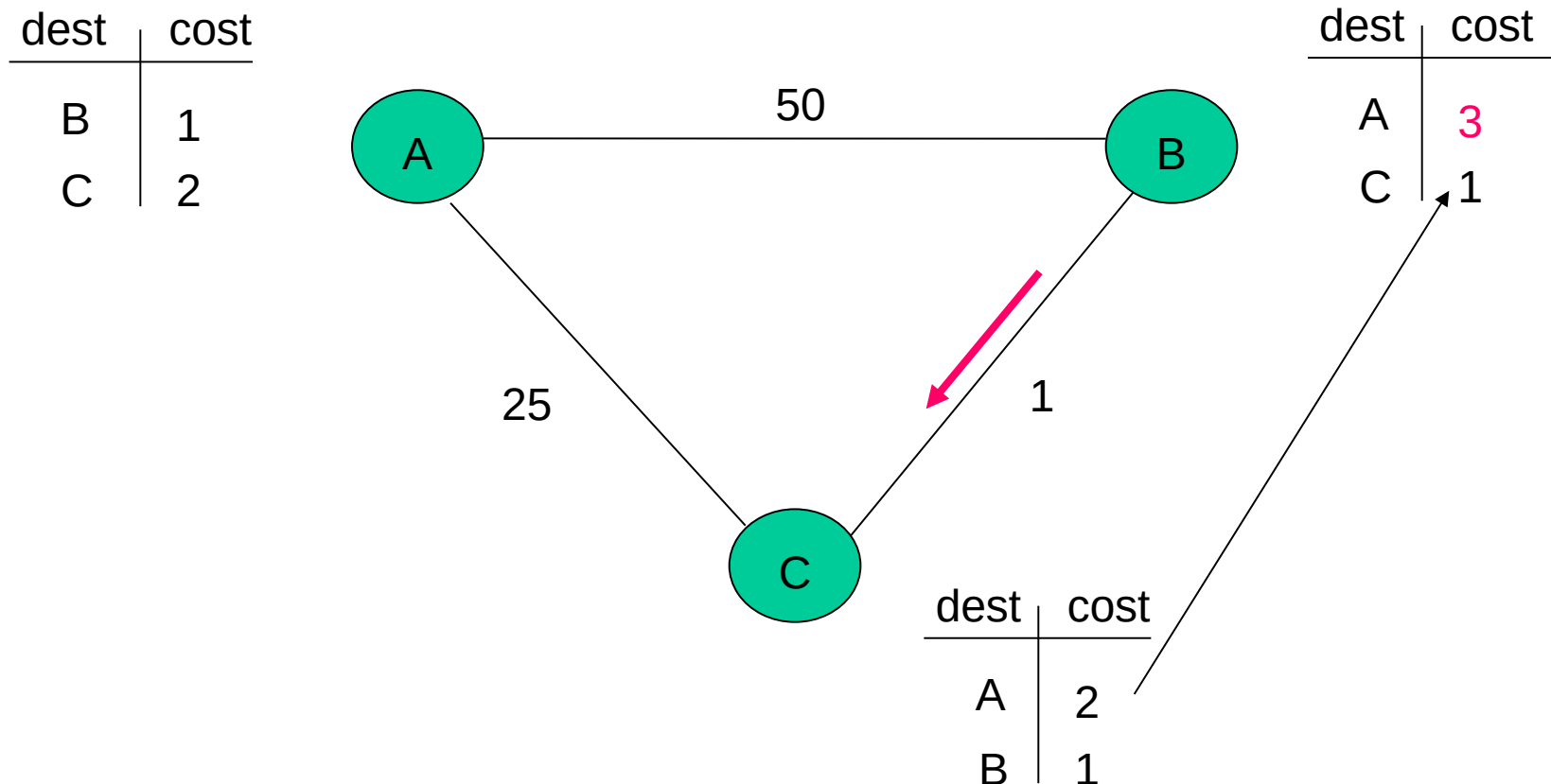
B aggiorna le distanze per A (tenendo però conto dei dati di C, che include il passaggio per B)



B aggiorna distanze verso A



B aggiorna le distanze per A (tenendo però conto dei dati di C, che include il passaggio per B) e le comunica a C

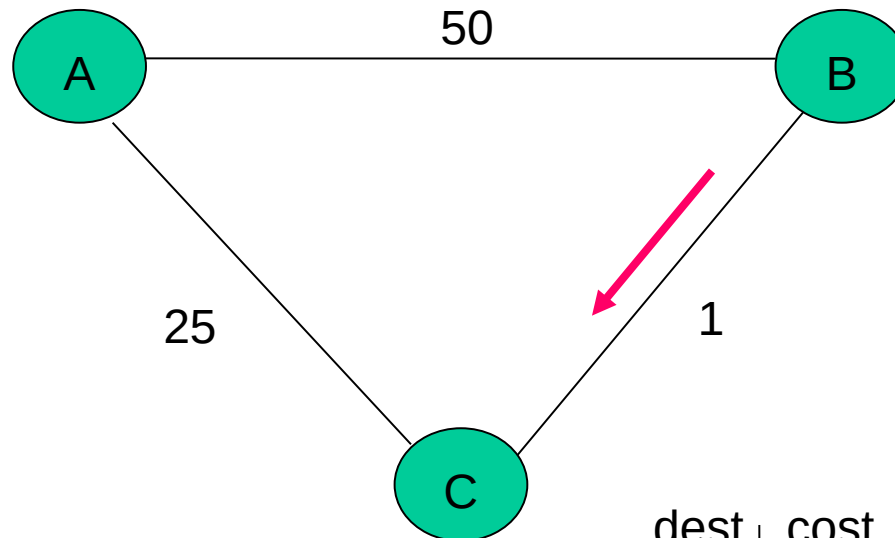


B manda il vettore di distanze a C



A sua volta C aggiorna le distanze per A
(tenendo conto però di continuare a passare da B)

dest	cost
B	1
C	2



dest	cost
A	3
C	1

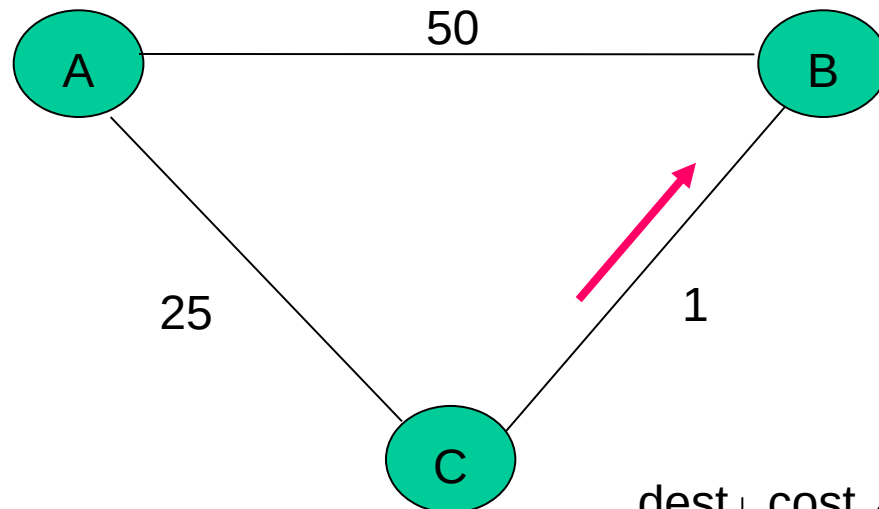
dest	cost
A	4
B	1

C manda il vettore di distanze a B



A sua volta B aggiorna le distanze per A
(tenendo conto però di continuare a passare da C)

dest	cost
B	1
C	2



dest	cost
A	5
C	1

dest	cost
A	4
B	1

Come si crea l'effetto rimbalzo



- La distanza diretta da B verso A **cresce molto**
 - Quindi, B sceglie C come prossimo hop per A
 - Ma... il **percorso implicito** da C verso A include B!
 - Le tabelle di B e C si aggiornano gradualmente, ma si crea un **loop** che proseguirà fino a quando C considererà il proprio percorso verso A attraverso B minore di 25
 - Poi finalmente sceglierà il percorso diretto
- **Un pacchetto che arrivi a B o a C durante l'esistenza del loop rimbalzerà tra questi due nodi**

- **Inversione avvelenata**

- Se B raggiunge A attraverso C, B avvertirà C che la sua distanza verso A è infinita (mentendo - sa di poter instradare i pacchetti tramite C, ma serve a evitare effetto rimbalzo)
- Così C non tenterà di instradare per A attraverso B
- Non funziona per cicli che coinvolgono 3 o più nodi

- **Split Horizon**

- Bisogna differenziare i vettori di distanze inviati ai nodi adiacenti: il vettore di B inviato a C non conterrà le destinazioni raggiungibili tramite C
- Obiettivo: “Se B raggiunge A attraverso C, non ha senso per C cercare di raggiungere A attraverso B”

Algoritmi Link State vs. Distance Vector

DV vs. LS



- LS: le informazioni sui vicini sono passate a tutti
- DV: tutto quello che si sa è propagato solo ai vicini
- **Dimensione dei messaggi**
 - LS: piccola
 - DV: potenzialmente grande
- **Numero di messaggi**
 - LS: molto grande, di tipo $O(n)$, dove n sono i nodi del grafo
 - DV: piccolo in quanto comunicazioni solo ai vicini
- **Velocità di convergenza**
 - LS: veloce (e senza rischi di cicli)
 - DV: veloce se si usano aggiornamenti periodici abbastanza frequenti (però, troppo frequenti sono a rischio di instabilità)
- **Requisito di memorizzazione**
 - LS: molto alto → si mantiene l'intera topologia del grafo
 - DV: basso → si mantiene solo lo stato dei vicini

- **Robustezza**
 - LS: calcolo dei percorsi effettuato in maniera indipendente da ogni nodo → protezione contro guasti ai router
 - DV: calcolo dei percorsi basato sui calcoli degli altri router → il calcolo sbagliato di un router può essere propagato a gran parte della rete
- **Nessuno dei due è superiore: entrambi usati in Internet**
 - Gli algoritmi di tipo **Link state (centralizzati)** tendono ad essere utilizzati all'interno degli **AS**
 - Gli algoritmi di tipo **Distance vector (distribuiti)** sono utilizzati per il routing **tra AS**

"E' vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

E' inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia."